

Python: OpenCV findHomography Inputs

Last Updated : 25 Jun, 2024

OpenCV, the popular open-source computer vision and machine learning library, offers a wide range of tools for image processing and computer vision tasks. One of the critical functions in the context of image alignment and perspective transformation is `findHomography`. This function is used to find the transformation matrix that maps points from one plane to another. Understanding the inputs required for this function is crucial for its effective utilization. This article explores the essential inputs for the `findHomography` function in [OpenCV](#).

What is `findHomography`?

The `findHomography` function in OpenCV calculates a homography matrix that describes the perspective transformation between two sets of points. This matrix is fundamental in tasks such as image stitching, perspective correction, and object recognition.

The basic syntax for the function is:

```
retval, mask = cv2.findHomography(srcPoints, dstPoints,
method[, ransacReprojThreshold[, mask[, maxIters[,
confidence]]]])
```

Let's break down each input parameter in detail.

Essential Inputs

`srcPoints`:

- **Type:** `numpy.ndarray` or `list`
- **Description:** This parameter represents the source points in the source image. These points are usually coordinates (x, y) in the original image. The `srcPoints` array should have the shape of `Nx2`, where `N` is the number of points. Each point corresponds to

a feature in the source image that matches a feature in the destination image.

Example:

```
srcPoints = np.array([[100, 100], [150, 100], [150, 150], [100, 150]])
```

dstPoints:

- **Type:** `numpy.ndarray` or `list`
- **Description:** This parameter represents the destination points in the destination image. These points correspond to the `srcPoints` after the perspective transformation. Similar to `srcPoints`, the `dstPoints` array should also have the shape of `Nx2`

Example:

```
dstPoints = np.array([[200, 200], [250, 200], [250, 250], [200, 250]])
```

method:

- **Type:** `int`
- **Description:** This parameter specifies the method used to compute the homography matrix. There are several methods available:
 - **0 or `cv2.RANSAC`:** RANSAC-based robust method. It is widely used due to its robustness to outliers.
 - **`cv2.LMEDS`:** Least-Median robust method.
 - **`cv2.RHO`:** RHO method, another robust method.

Example:

```
method = cv2.RANSAC
```

ransacReprojThreshold (optional):

- **Type:** float
- **Description:** This parameter is used when the RANSAC method is selected. It specifies the maximum allowed reprojection error to treat a point pair as an inlier. The smaller the value, the more stringent the criteria.

Example:

```
ransacReprojThreshold = 5.0
```

- **mask** (optional):
- **Type:** `numpy.ndarray`
- **Description:** This parameter is an optional output mask. It indicates which points are inliers (1) and which are outliers (0) when a robust method is used.

Example:

```
mask = np.zeros((len(srcPoints), 1), dtype=np.uint8)
```

maxIters (optional):

- **Type:** int
- **Description:** This parameter is used to specify the maximum number of RANSAC iterations. It is optional and only used with robust methods.

Example:

```
confidence = 0.995
```

confidence (optional):

- **Type:** float
- **Description:** This parameter is the confidence level, which is used to specify the desired confidence level for the estimated

transformation. It is optional and only used with robust methods.

Example:

```
confidence = 0.995
```

Practical Example

Here is a practical example demonstrating how to use the findHomography function with some sample points:

```
import cv2
import numpy as np

# Define source and destination points
srcPoints = np.array([[100, 100], [150, 100], [150, 150], [100, 150]], dtype=np.float32)
dstPoints = np.array([[200, 200], [250, 200], [250, 250], [200, 250]], dtype=np.float32)

# Compute homography matrix
H, mask = cv2.findHomography(srcPoints, dstPoints, cv2.RANSAC, 5.0)

# Print the resulting homography matrix
print("Homography Matrix:")
print(H)
```

Output

```
Homography Matrix:
[[ 1.  0. 100.]
 [ 0.  1. 100.]
 [ 0.  0.  1.]]
```

In this example, the source points and destination points are defined, and the findHomography function is called with the RANSAC method. The resulting homography matrix H is printed, which can then be used to warp images or perform other perspective transformations.

Conclusion

The findHomography function in OpenCV is a powerful tool for calculating the transformation matrix between two sets of points in

different planes. Understanding its inputs, such as source and destination points, method, and optional parameters, is crucial for its effective application in various computer vision tasks. By leveraging this function, developers can achieve tasks like image alignment, stitching, and perspective correction with high precision.